30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 3

Architecture Patterns — MVC → MVVM → MVI, UDF & Clean Architecture



Esmaeeil Moradi
@esmaeeil-moradi700

Why architecture is Day 3

Architecture questions show the interviewer **how you think**, not which libraries you know. The 2026 answer is no longer "MVP vs MVVM":

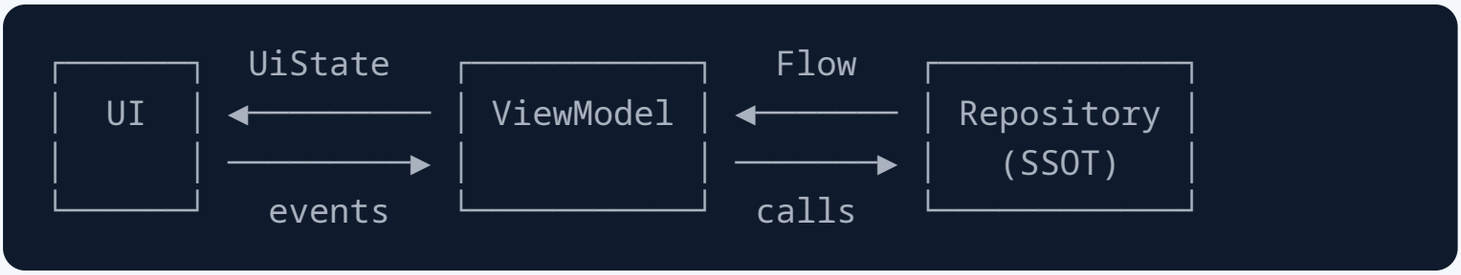
- Google's architecture guide: **UI layer → (domain) → data layer**
- **Unidirectional data flow** and a **single source of truth**
- MVVM with one immutable `UiState` — which is already most of MVI
- Clean Architecture ideas, applied **pragmatically**
- The same architecture shared across platforms with KMP

Level	Questions
Junior	MVC / MVP / MVVM · Why ViewModel · The layers
Mid	UDF & SSOT · Modeling UI state · One-off events · MVVM vs MVI
Senior	Repository as SSOT · Use cases · Testing the architecture
Architect	Clean Architecture done right · Governance as code · KMP



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: state down, events up



- **State flows down:** the UI only renders what it's given
- **Events flow up:** the UI reports what happened, it never changes state itself
- **Single source of truth:** each piece of data has exactly one owner allowed to change it

THE ONE IDEA BEHIND EVERY PATTERN TODAY

Whatever the name — MVVM, MVI, Clean — a good answer always comes back to: *who owns this state, and which direction does data flow?*



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 3

Junior

MVC / MVP / MVVM · Why ViewModel · The layers



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. MVC, MVP, MVVM — what each one fixes

Pattern	Presentation logic lives in	View is updated by	Weak spot on Android
MVC	Controller — usually the Activity itself	Controller pushing to the view	"God Activity": untestable, dies on rotation
MVP	Presenter behind a `View` interface	Presenter calling `view.showX()`	Doesn't survive rotation; manual attach/detach
MVVM	ViewModel exposing observable state	View observing a `StateFlow`	Without discipline: many streams, scattered state

All three separate UI from logic. The difference is **who knows whom**: the presenter holds a reference to the view; the ViewModel knows **nothing** about the view.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"MVVM means Data Binding." — **No.** Data Binding was one way to observe a ViewModel. In 2026 the view is Compose — or Views collecting a `StateFlow` with `repeatOnLifecycle`.

INTERVIEW TRAP

"In MVC the Activity is the View." — On Android it ends up being View **and** Controller at once. That's exactly how the God Activity is born.

FOLLOW-UP

"Why did Android move from MVP to MVVM?" — Jetpack `ViewModel` survives configuration changes and comes with `viewModelScope`. A presenter has neither, so every MVP codebase reinvented retention and attach/detach logic.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Why ViewModel fits Android

```
class ProfileViewModel(
    repo: ProfileRepository,
    handle: SavedStateHandle,
) : ViewModel() {
    private val userId: String = checkNotNull(handle["userId"])

    val uiState: StateFlow<ProfileUiState> =
        repo.observe(userId)
            .map { ProfileUiState.Ready(it) }
            .stateIn(
                scope = viewModelScope,
                started = SharingStarted.WhileSubscribed(5_000),
                initialValue = ProfileUiState.Loading,
            )
}
```

- Survives **configuration changes**; cleared only when its owner is gone for good (`onCleared()`, `viewModelScope` cancelled)
- Scoped to an owner: Activity, Fragment or navigation back-stack entry



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. The limits you must know

- A ViewModel **doesn't survive process death**. The keys needed to rebuild the screen go into `SavedStateHandle`; the data itself comes back from the repository.
- It holds **state and logic**, never views.

INTERVIEW TRAP

Holding a `Context`, `View` or `Activity` in a ViewModel leaks the whole old screen after rotation. Need resources or system services? Put them behind a repository and inject it. `AndroidViewModel` is usually a smell.

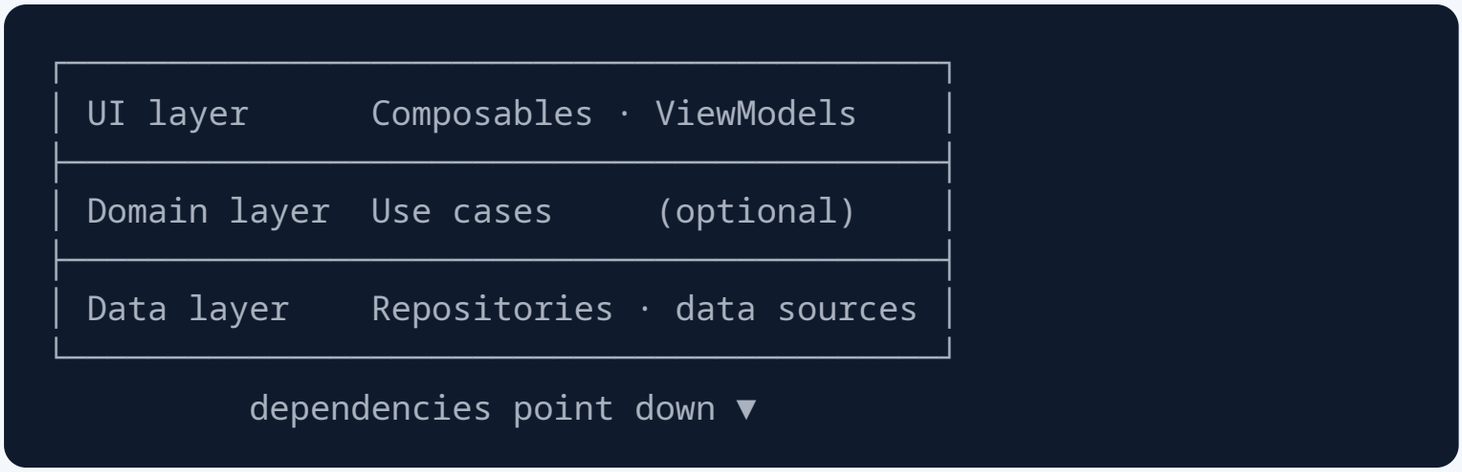
FOLLOW-UP

"Who creates the ViewModel with constructor parameters?" — A `ViewModelProvider.Factory` — in practice generated by DI (Hilt, Koin) or the `viewModelFactory { initializer { … } }` DSL.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. The layers of a modern Android app



- **UI layer:** renders state and sends events. State holders (ViewModels) turn data into UI state.
- **Domain layer (optional):** business logic reused by several ViewModels.
- **Data layer:** repositories expose data and own its rules; each data source talks to one source (Room, Ktor/Retrofit, DataStore).

INTERVIEW TRAP

"Every app needs a domain layer." — Google's guide makes it **optional**. Add it when logic is reused or complex enough to deserve its own tests.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 3

Mid-Level

UDF & SSOT · Modeling UI state · One-off events · MVVM vs MVI



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. UDF and SSOT in code

```
data class SearchUiState(
    val query: String = "",
    val results: List<Item> = emptyList(),
    val isLoading: Boolean = false,
)

class SearchViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(SearchUiState())
    val uiState: StateFlow<SearchUiState> = _uiState.asStateFlow()

    fun onQueryChange(query: String) {
        _uiState.update { it.copy(query = query) }
    }
}
```

- Only the ViewModel **writes** the state; everyone else reads a read-only `StateFlow`
- `update {}` is atomic (compare-and-set) — safe when several coroutines change state at once



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. The UI side: stateful route, stateless screen

```
@Composable
fun SearchRoute(viewModel: SearchViewModel = hiltViewModel()) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    SearchScreen(state, onQueryChange = viewModel::onQueryChange)
}

@Composable
fun SearchScreen(
    state: SearchUiState,
    onQueryChange: (String) -> Unit,
) { /* stateless: easy to preview and test */ }
```

- The **route** connects to the ViewModel; the **screen** is a pure function of its parameters
- `collectAsStateWithLifecycle()` stops collecting when the app goes to the background

INTERVIEW TRAP

Exposing `MutableStateFlow` publicly: now any screen can write it, and there is no single source of truth anymore.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Modeling UI state

Sealed interface — when the states are mutually exclusive:

```
sealed interface FeedUiState {
    data object Loading : FeedUiState
    data class Success(val posts: List<Post>) : FeedUiState
    data class Error(val message: String) : FeedUiState
}
```

Data class — when parts coexist (content + refreshing + message):

```
data class FeedUiState(
    val posts: List<Post> = emptyList(),
    val isRefreshing: Boolean = false,
    val userMessage: String? = null,
)
```

MAKE IMPOSSIBLE STATES IMPOSSIBLE

If "loading" and "error" can never happen together, don't model them as two booleans that can both be `true`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. One stream per screen

```
val uiState: StateFlow<FeedUiState> = combine(
    repo.posts, repo.isRefreshing, userMessages,
) { posts, refreshing, message ->
    FeedUiState(posts, refreshing, message)
}.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = FeedUiState(),
)
```

- One `StateFlow<UiState>` per screen: one render path, one thing to test
- `WhileSubscribed(5_000)`: upstream stops 5 s after the UI leaves, but a rotation doesn't restart it
- Immutable state (`val`, read-only `List`) — Compose can safely skip recompositions

INTERVIEW TRAP

"One giant state for the whole app." — Scope state to the screen. App-wide state belongs in repositories.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. One-off events: navigation, snackbar, "payment done"

Approach	Risk
`SharedFlow(replay = 0)`	Event is **dropped** if nobody collects at that moment (rotation, background)
`Channel` + `receiveAsFlow()`	Buffered, but can still be lost unless collected on `Dispatchers.Main.immediate`
`SingleLiveEvent`	Legacy LiveData hack; one observer only
Event as state (Google's guidance)	Survives rotation; the UI tells the ViewModel when it's handled

INTERVIEW TRAP

"A `Channel` guarantees delivery." — Not across process death, and not when collected off `Main.immediate`. For outcomes that must never be lost (a payment), use **state + persistence**.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Events as state

```
data class CheckoutUiState(
    val isPaying: Boolean = false,
    val paymentResult: PaymentResult? = null,
)

// ViewModel
fun onPaymentResultHandled() {
    _uiState.update { it.copy(paymentResult = null) }
}

// UI
state.paymentResult?.let { result ->
    LaunchedEffect(result) {
        onNavigateToReceipt(result.id)
        viewModel.onPaymentResultHandled()
    }
}
```

The result stays in state until the UI **confirms** it handled it — rotation, background or a slow collector can't lose it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. MVI in practice

MVI = MVVM + one immutable state + every input as a typed intent + a pure reducer.

```
sealed interface CartIntent {
    data class Add(val item: Item) : CartIntent
    data class Remove(val id: String) : CartIntent
    data object Checkout : CartIntent
}

fun CartState.reduce(intent: CartIntent): CartState =
    when (intent) {
        is CartIntent.Add -> copy(items = items + intent.item)
        is CartIntent.Remove ->
            copy(items = items.filter { it.id != intent.id })
        CartIntent.Checkout -> copy(isCheckingOut = true)
    }
```

The reducer is pure: same state + same intent → same result. Side effects (network, navigation) run **outside** it and come back as new intents.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. MVVM or MVI?

	MVVM + UDF	MVI
Input	Functions (`onQueryChange()`)	One `onIntent(intent)` entry point
State	One `UiState` (recommended)	One `UiState` (required)
Logic	Inside ViewModel functions	Pure reducer + separate effects
Best for	Most screens	Complex, event-heavy screens: editors, trading, multi-step flows
Cost	Low	More types, more ceremony

MY TAKE

In 2026 the gap is small: MVVM with a single `UiState` and UDF is already most of MVI. Choose MVI when you need the reducer's guarantees — auditability, replaying intents in tests — not because it's trendy.

INTERVIEW TRAP

"MVI needs a framework." — It's a discipline. A sealed interface and a `when` are enough; Orbit or Mavericks are optional.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 3



Senior

Repository as SSOT · Use cases · Testing the architecture



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. The repository as single source of truth

```
class ArticleRepository(
    private val dao: ArticleDao,
    private val api: ArticleApi,
) {
    // The database is the single source of truth
    fun observeArticles(): Flow<List<Article>> =
        dao.observeAll().map { rows -> rows.map { it.toDomain() } }

    suspend fun refresh(): Result<Unit> = try {
        val remote = api.fetchArticles()
        dao.upsertAll(remote.map { it.toEntity() })
        Result.success(Unit)
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Result.failure(e)
    }
}
```

The UI observes the **database**; the network only **writes into it**. One truth, works offline, no "which copy is right?".



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Rules for a good repository

- **Map at the boundary:** DTO → domain model → UI model. The UI never sees an `@Entity` or a network DTO.
- `Flow` for streams, `suspend` for one-shot operations. **Never** expose mutable state.
- **Main-safe:** callers never switch threads. Room and Retrofit/Ktor `suspend` calls already are; move only heavy CPU work (large parsing, sorting) to a background dispatcher.

INTERVIEW TRAP

`runCatching { }` in a coroutine also catches `CancellationException` and breaks structured cancellation. Rethrow it, or catch specific exceptions.

INTERVIEW TRAP

Caching the same data in the ViewModel **and** in Room — now there are two sources of truth, and they will disagree.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Use cases: when they earn their place

```
class GetLatestNewsWithAuthorsUseCase(
    private val news: NewsRepository,
    private val authors: AuthorsRepository,
) {
    operator fun invoke(): Flow<List<ArticleWithAuthor>> =
        combine(news.latest(), authors.byId()) { articles, byId ->
            articles.map { a ->
                ArticleWithAuthor(a, byId.getValue(a.authorId))
            }
        }
}
```

- Add one when logic is **reused** by several ViewModels, or **complex** enough to test on its own
- One job, verb-named (`GetLatestNews…`, `ValidateIban…`), pure Kotlin, no Android imports

INTERVIEW TRAP

Pass-through use cases — a `GetUserUseCase` that only calls `repo.getUser()`. Ceremony with no value; the ViewModel can call the repository directly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Testing the architecture — setup

```
class MainDispatcherRule(
    val dispatcher: TestDispatcher = UnconfinedTestDispatcher(),
) : TestWatcher() {
    override fun starting(description: Description) {
        Dispatchers.setMain(dispatcher)
    }
    override fun finished(description: Description) {
        Dispatchers.resetMain()
    }
}
```

- `viewModelScope` runs on `Dispatchers.Main` — replace it in unit tests
- Inject every other dispatcher, so tests can control it
- **Fakes over mocks** for repositories: real behavior, no brittle `verify` chains



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Testing a ViewModel

```
class FeedViewModelTest {
    @get:Rule val mainRule = MainDispatcherRule()

    @Test fun showsPostsFromRepository() = runTest {
        val repo = FakeFeedRepository(posts = listOf(post1))
        val viewModel = FeedViewModel(repo)

        val dispatcher = UnconfinedTestDispatcher(testScheduler)
        backgroundScope.launch(dispatcher) {
            viewModel.uiState.collect()
        }

        val state = viewModel.uiState.value
        assertEquals(listOf(post1), state.posts)
    }
}
```

- `stateIn(WhileSubscribed)` only runs with a collector — start one in `backgroundScope`
- Test the stateless screen separately with `createComposeRule()`

INTERVIEW TRAP

`Thread.sleep()` or real delays in tests. Use the test scheduler: `advanceUntilIdle()`, `advanceTimeBy()`.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 3

→

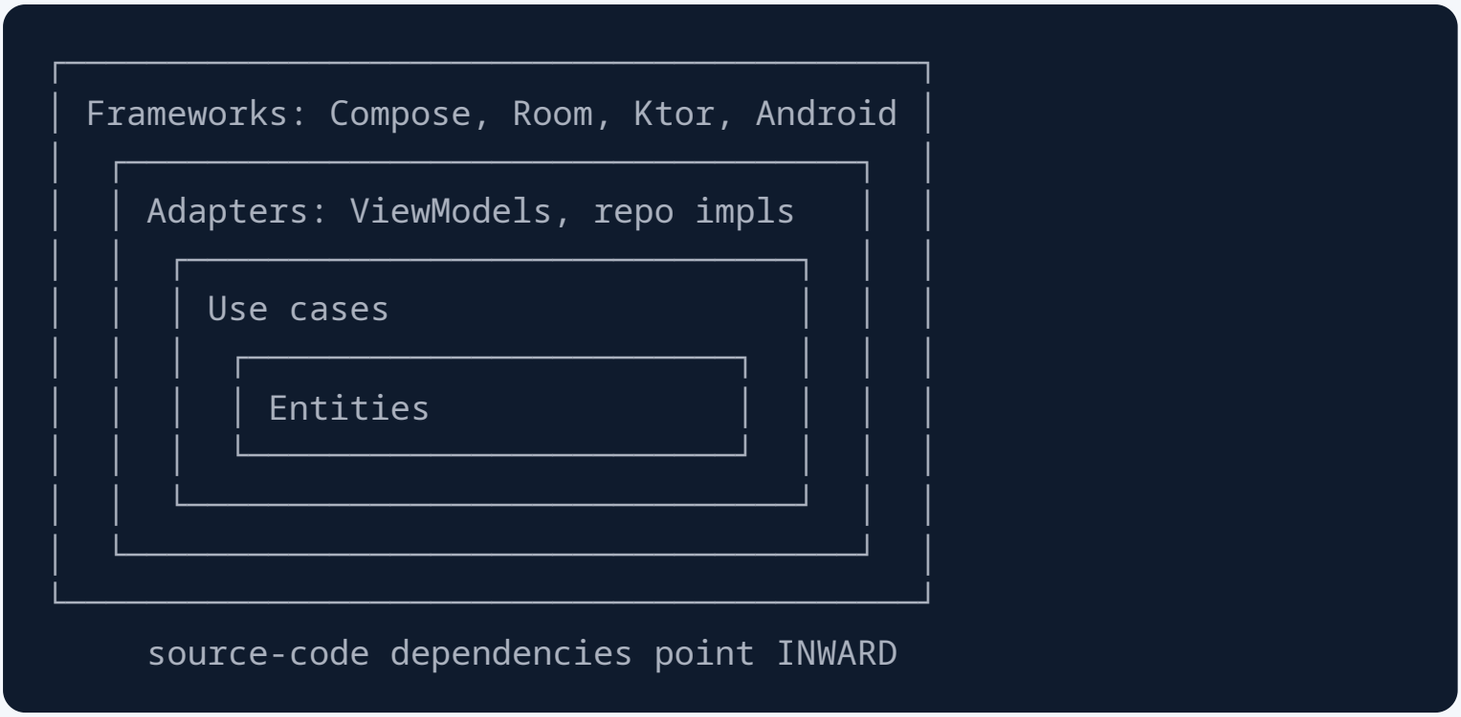
Architect

Clean Architecture done right · Governance as code · KMP



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Clean Architecture: the dependency rule



- Inner circles know **nothing** about outer ones. Outer layers depend on inner layers — never the reverse.
- **Dependency inversion:** the domain declares `interface ArticleRepository`; the data layer implements it; DI wires them.

INTERVIEW TRAP

"Outer layers must not depend on inner layers." — Backwards. It's the **inner** layers that must never depend on the outer ones.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Where it pays off — and where it hurts

Pays off	Hurts
Large teams, long-lived product, logic shared across platforms	Small apps, prototypes, simple CRUD screens
Business rules you must test in isolation (fintech, health)	Mapping the same model three times with no logic in between
Swapping frameworks (Views → Compose, Retrofit → Ktor)	A module per layer per feature for five screens

- Structure by **feature first**, layers inside (`feature/checkout/{ui, domain, data}`) — not global `ui/`, `domain/`, `data/` folders for the whole app.

FOLLOW-UP

"How would you move a legacy MVP app to this?" — Strangler approach: new features in the target architecture; extract repositories first (biggest win: SSOT + testability); then replace presenters with ViewModels screen by screen.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Architecture governance as code

Rules that live only in a wiki erode. Encode them as **fitness functions** that fail the CI build:

```
class ArchitectureTest {
    @Test fun layersRespectDependencyRule() {
        Konsist.scopeFromProduction().assertArchitecture {
            val domain = Layer("Domain", "com.app.domain..")
            val data = Layer("Data", "com.app.data..")
            val ui = Layer("UI", "com.app.ui..")

            domain.dependsOnNothing()
            data.dependsOn(domain)
            ui.dependsOn(domain)
        }
    }

    @Test fun viewModelsLiveInUiLayer() {
        Konsist.scopeFromProduction()
            .classes()
            .withNameEndingWith("ViewModel")
            .assertTrue { it.resideInPackage("..ui..") }
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. The governance toolbox

Tool	Enforces
Gradle module boundaries (`api` / `impl`, `implementation` over `api`)	Who can see what — at compile time
Konsist / ArchUnit	Layer, naming and package rules as unit tests
Custom Android Lint checks, detekt, ktlint	Code-level rules and style
Dependency Analysis Gradle Plugin	Unused and misdeclared dependencies
ADRs in the repository	*Why* a decision was made

MY TAKE: SOLID CORE, FLEXIBLE EDGES

Strict rules at module boundaries and in shared core modules; teams stay free inside their feature (MVVM or MVI per screen). Tools are the good cop; ADRs and guilds explain the *why*.

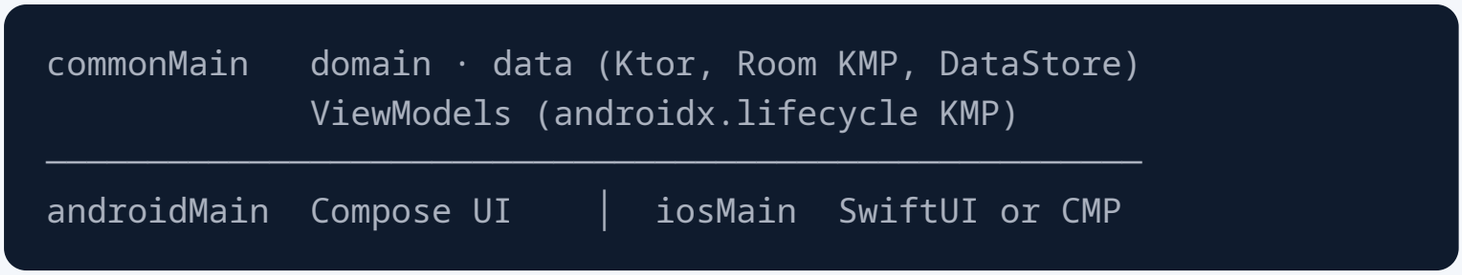
INTERVIEW TRAP

Governance only through code review — it doesn't scale, and it makes the architect a bottleneck.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. One architecture, many platforms: KMP



- Jetpack `ViewModel` and `viewModelScope` are multiplatform since Lifecycle 2.8; Room supports KMP since 2.7
- Share **data + domain** first (highest value, lowest risk), then ViewModels; share UI (Compose Multiplatform) only if it fits the team
- Keep platform APIs behind interfaces (`expect` / `actual` or DI)

INTERVIEW TRAP

"KMP means the iOS app is written in Kotlin." — Not necessarily. Shared logic with native UI is the most common, lowest-risk setup.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. A shared ViewModel

```
// commonMain — compiled for Android and iOS
class ArticlesViewModel(
    private val repo: ArticleRepository,
) : ViewModel() {
    val articles: StateFlow<List<Article>> =
        repo.observeArticles().stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = emptyList(),
        )

    fun refresh() {
        viewModelScope.launch { repo.refresh() }
    }
}
```

- The **same** UDF rules apply on every platform: one state, events up, repository as SSOT
- The architecture you chose on Android becomes the contract for the whole product

SENIOR ANSWER IN ONE LINE

"Pick the pattern per screen, but fix the rules for the whole app: one state per screen, data flows one way, every piece of data has one owner, and dependencies point inward."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 3 · ARCHITECT

Day 3 — cheat sheet

- MVC → MVP → MVVM: each moves logic further from the view. In 2026: **MVVM + UDF, one `UiState` per screen**.
- ViewModel survives rotation, **not** process death → `SavedStateHandle` for keys.
- Layers: UI → (domain, optional) → data. The repository is the single source of truth.
- State down, events up. Expose `StateFlow`, never `MutableStateFlow`.
- Sealed interface for exclusive states; data class for coexisting parts.
- One-off events: model them as state and acknowledge; `SharedFlow(replay = 0)` can drop them.
- MVI = one state + typed intents + a pure reducer — a discipline, not a library.
- Use cases only when reused or complex — no pass-through use cases.
- Test ViewModels with fakes, `runTest` and a Main dispatcher rule.
- Clean Architecture: dependencies point **inward** — enforce it with Gradle modules and Konsist in CI.

Day 4 is next → Coroutines. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700